

# Elevate

Sistema de gestión gastronómica — Tienda web + Punto de venta + ERP financiero

Documentación técnica de arquitectura

Next.js 16 · React 19 · TypeScript · PostgreSQL · Prisma

Documento generado para revisión técnica · Julio 2026

---

## Índice

---

1. Visión general: qué es Elevate
2. Stack tecnológico y justificación
3. Arquitectura general (el "mapa mental")
4. Estructura de carpetas explicada y justificada
5. Dónde está el Frontend, el Backend y la Base de Datos
6. Cómo interactúan Frontend y Backend
7. Webhooks e integraciones externas
8. Ciberseguridad: capas, dónde y cómo
9. Flujos completos de ejemplo
10. Despliegue (deploy)
11. Riesgos detectados y recomendaciones

# 1. Visión general: qué es Elevate

---

Elevate es una plataforma **todo-en-uno** para un negocio de comida. Reúne tres sistemas que normalmente son productos separados:

| Sub-sistema              | Para quién    | Qué hace                                                                                                                                      |
|--------------------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tienda / Menú web</b> | Cliente final | Ver el catálogo por marca, armar pedido, elegir delivery o recojo.                                                                            |
| <b>Caja / POS</b>        | Cajero        | Vender en mostrador (efectivo/QR), abrir y cerrar turno, gastos, fiados, entregar pedidos web.                                                |
| <b>Panel Admin / ERP</b> | Dueño / Admin | Productos, insumos, recetas, inventario, contabilidad, flujo de caja, cuentas por cobrar/pagar, auditoría, usuarios, rastreo de repartidores. |

Existe además un **envoltorio de escritorio** (Electron, `main.js`) que permite abrir la aplicación web como si fuera un programa nativo de Windows para la caja.

## 2. Stack tecnológico y justificación

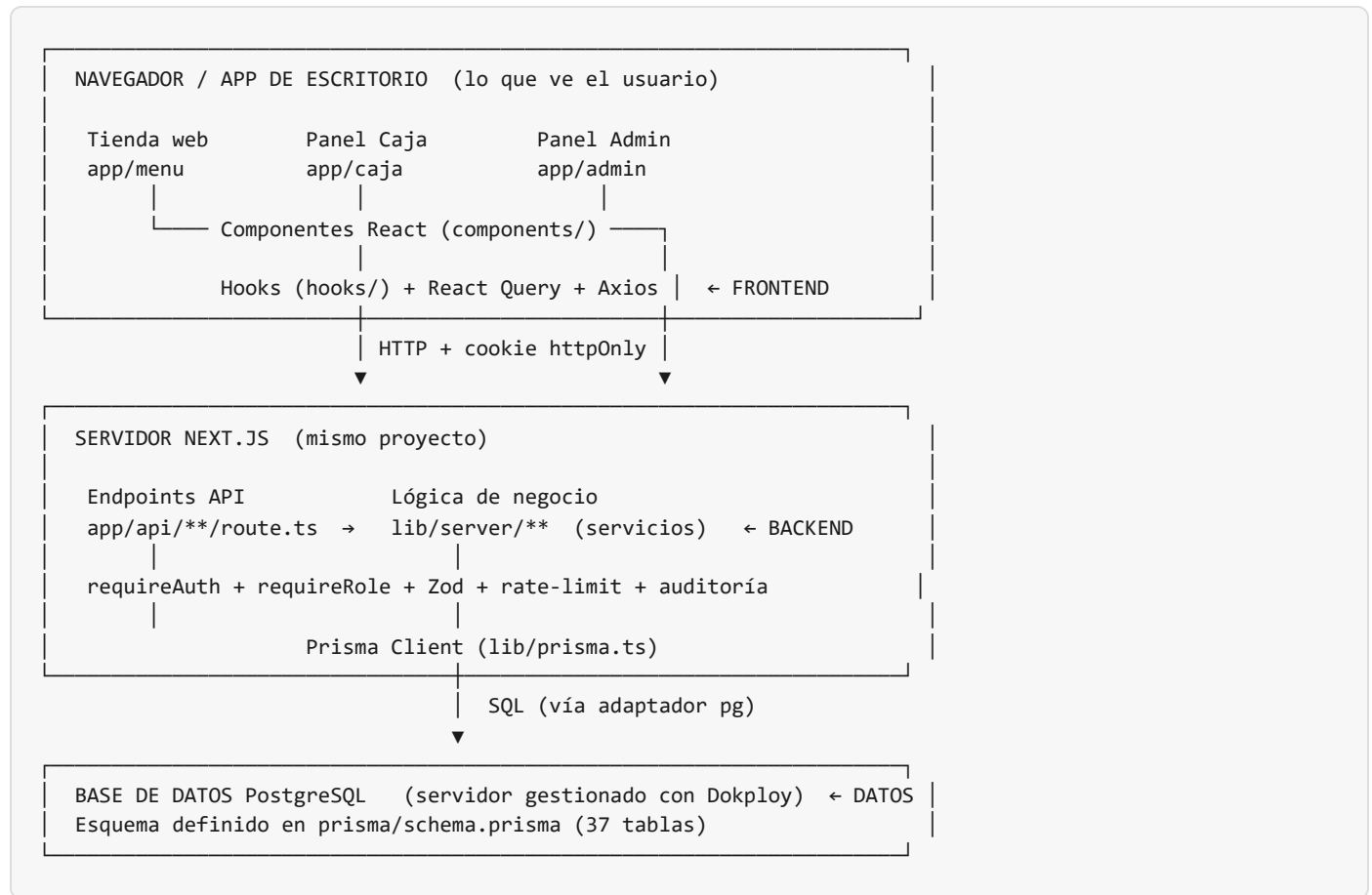
| Capa            | Tecnología                          | Por qué se eligió                                                                                                                    |
|-----------------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Framework       | <b>Next.js 16</b> (App Router)      | Un solo proyecto contiene frontend Y backend. Menos infraestructura, un solo despliegue, código compartido entre cliente y servidor. |
| UI              | <b>React 19 + TypeScript</b>        | Componentes reutilizables y tipado estático que atrapa errores antes de ejecutar.                                                    |
| Estilos         | <b>Tailwind CSS v4 + PrimeReact</b> | Estilos rápidos y consistentes + componentes ricos ya hechos (tablas, diálogos).                                                     |
| Animación       | <b>Framer Motion</b>                | Transiciones fluidas en la tienda (experiencia "premium").                                                                           |
| Datos (cliente) | <b>TanStack React Query + Axios</b> | Maneja peticiones al servidor, caché, refresco automático y estados de carga/error sin código repetitivo.                            |
| ORM             | <b>Prisma</b> (+ adapter-pg)        | Traduce entre el código TypeScript y las tablas SQL con tipado seguro. Evita escribir SQL a mano y previene inyección.               |
| Base de datos   | <b>PostgreSQL</b>                   | Base relacional robusta, ideal para datos financieros con integridad referencial (claves foráneas, transacciones ACID).              |
| Auth            | <b>JWT + bcryptjs</b>               | Autenticación propia con control total sobre usuarios y roles en la base local.                                                      |
| Validación      | <b>Zod</b>                          | Valida y limpia toda entrada del usuario en el servidor antes de tocar la base.                                                      |
| Mapas           | <b>Leaflet</b>                      | Rastreo de repartidores en tiempo real.                                                                                              |
| Gráficos        | <b>Recharts</b>                     | Estadísticas visuales del dashboard.                                                                                                 |
| Archivos        | <b>Vercel Blob</b>                  | Almacenar imágenes de productos fuera de la base de datos.                                                                           |
| Escritorio      | <b>Electron</b>                     | Empaquetar la web como app de Windows para la caja.                                                                                  |

### La decisión clave: un monolito full-stack

La justificación más importante de toda la arquitectura es que **frontend y backend viven en el mismo proyecto Next.js**. En lugar de tener un servidor de API separado (Node/Express) y una app de React aparte, Next.js permite que ambos convivan. Para un negocio de este tamaño esto significa: menos servidores que pagar y mantener, un solo despliegue, y tipos de datos compartidos entre lo que el navegador envía y lo que el servidor recibe.

### 3. Arquitectura general (el "mapa mental")

Todo el sistema sigue este flujo de tres capas. Léelo de arriba (usuario) hacia abajo (base de datos):



La regla de oro es **de una sola dirección**: el navegador nunca habla directo con la base de datos. Siempre pasa por un endpoint de API que verifica identidad, permisos y validez de los datos antes de dejar que Prisma toque PostgreSQL.

## 4. Estructura de carpetas explicada y justificada

### Carpetas de aplicación

| Carpeta                                    | Rol          | Qué contiene y por qué                                                                                                                                                                                                                                  |
|--------------------------------------------|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| app/                                       | FRONT + BACK | El corazón de Next.js App Router. Cada subcarpeta es una <i>ruta</i> . Aquí conviven las páginas (frontend) y los endpoints ( <code>app/api/</code> , backend). Se organiza así porque Next.js mapea carpetas a URLs automáticamente.                   |
| app/menu/ ,<br>app/login/ ,<br>app/driver/ | FRONT        | Páginas públicas: catálogo por marca, inicio de sesión, y vista del repartidor (accedida por token).                                                                                                                                                    |
| app/caja/                                  | FRONT        | Las 13 pantallas del cajero (venta, movimientos, apertura, cierre, gasto, deudores, historial, entregar, pedidos).                                                                                                                                      |
| app/admin/                                 | FRONT        | Las 25 pantallas del panel administrativo/ERP.                                                                                                                                                                                                          |
| app/api/                                   | BACK         | <b>Todo el backend HTTP.</b> 93 endpoints ( <code>route.ts</code> ) agrupados por dominio: <code>auth</code> , <code>caja</code> , <code>pedidos</code> , <code>productos</code> , <code>admin/*</code> , etc.                                          |
| components/                                | FRONT        | Componentes React reutilizables, divididos por área: <code>admin/</code> , <code>caja/</code> , <code>shop/</code> (tienda) y <code>ui/</code> (piezas genéricas: botones, píldoras de método de pago, diálogos).                                       |
| hooks/                                     | FRONT        | "Ganchos" de React que encapsulan la comunicación con el backend (ej. <code>caja.ts</code> , <code>finanzas.ts</code> , <code>auth.ts</code> ). Un componente llama <code>useRegistrarVenta()</code> y no necesita saber cómo se hace la petición HTTP. |
| stores/                                    | FRONT        | Estado global compartido (categorías, reglas) para no repetir peticiones.                                                                                                                                                                               |

## Carpetas de servidor y datos

| Carpeta                   | Rol                        | Qué contiene y por qué                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>lib/</code>         | <b>BACK</b> + <b>FRONT</b> | Utilidades compartidas. <code>lib/prisma.ts</code> (conexión a la BD), <code>lib/auth.ts</code> (JWT/bcrypt), <code>lib/Guard.tsx</code> (protección de rutas en el cliente), <code>lib/providers.tsx</code> (React Query).                                                                                                                                                                             |
| <code>lib/server/</code>  | <b>BACK</b>                | <b>La lógica de negocio real.</b> Separada de los endpoints para poder reutilizarla y testearla. Subcarpetas por dominio: <code>auth/</code> (sesiones), <code>caja/</code> , <code>finanzas/</code> (contabilidad, flujo, gastos), <code>inventario/</code> (descuento de stock por recetas), <code>clientes/</code> , <code>audit/</code> (bitácora), <code>dto/</code> (esquemas Zod de validación). |
| <code>prisma/</code>      | <b>DB</b>                  | <code>schema.prisma</code> — la definición de las 37 tablas y sus relaciones. <code>migrations/</code> — el historial de cambios de la base. <code>seed.ts</code> — datos iniciales.                                                                                                                                                                                                                    |
| <code>public/</code>      | <b>FRONT</b>               | Archivos estáticos servidos tal cual: logo, íconos, imágenes.                                                                                                                                                                                                                                                                                                                                           |
| <code>data/</code>        | Datos                      | Datos semilla o de apoyo.                                                                                                                                                                                                                                                                                                                                                                               |
| <code>scripts/</code>     | 🔧 Ops                      | Utilidades de operación creadas para mantenimiento: respaldo ( <code>db-backup.mjs</code> ), limpieza ( <code>db-clean.mjs</code> ) y el SQL de reparación del módulo de fiado.                                                                                                                                                                                                                         |
| <code>backups/</code>     | 🔧 Ops                      | Respaldos JSON de la base (excluidos de git por contener datos de clientes).                                                                                                                                                                                                                                                                                                                            |
| <code>docs/</code>        | 📄 Docs                     | Documentación: guía de usuario final, este documento, y el handoff de base de datos.                                                                                                                                                                                                                                                                                                                    |
| <code>imagesExtra/</code> | 📄 Ref                      | Material de referencia del ERP "Paladar" que inspiró módulos de POS/finanzas.                                                                                                                                                                                                                                                                                                                           |

## Archivos de configuración en la raíz

| Archivo                                                                                       | Para qué sirve                                                                                   |
|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>package.json</code>                                                                     | Lista de dependencias y comandos ( <code>dev</code> , <code>build</code> , <code>start</code> ). |
| <code>prisma.config.ts</code>                                                                 | Configuración de Prisma para comandos de línea (migraciones, studio).                            |
| <code>next.config.ts</code>                                                                   | Ajustes de Next.js (imágenes, paquetes externos del servidor).                                   |
| <code>vercel.json</code>                                                                      | Instrucciones de despliegue en Vercel (instalación, generación del cliente Prisma y build).      |
| <code>docker-compose.yml</code>                                                               | Levanta un PostgreSQL local en Docker para desarrollo.                                           |
| <code>main.js</code>                                                                          | Punto de entrada de la app de escritorio Electron.                                               |
| <code>.env</code>                                                                             | Secretos y configuración: URL de la base, secreto JWT. ⚠️ <b>Nunca debe subirse a git.</b>       |
| <code>tsconfig.json</code> , <code>eslint.config.mjs</code> , <code>postcss.config.mjs</code> | Configuración de TypeScript, linter y procesamiento de CSS.                                      |

## 5. Dónde está el Frontend, el Backend y la Base de Datos

Esta es la pregunta más frecuente en un proyecto Next.js, porque todo vive junto. La separación es **por convención de carpetas**, no por proyectos distintos:

### FRONTEND — lo que corre en el navegador

- `app/**/page.tsx` — las páginas visibles (llevan `'use client'` cuando son interactivas).
- `components/` — piezas visuales reutilizables.
- `hooks/` — comunicación con el backend desde el navegador.
- `stores/`, `lib/Guard.tsx`, `lib/providers.tsx` — estado y protección de rutas del lado cliente.

### BACKEND — lo que corre en el servidor

- `app/api/**/route.ts` — los 93 endpoints HTTP. Es la *puerta* del backend.
- `lib/server/` — la *lógica* del backend (servicios de caja, finanzas, inventario, etc.).
- `lib/auth.ts`, `lib/server/auth/session.ts` — autenticación y sesiones.
- `lib/prisma.ts` — el único punto que habla con la base de datos.

### BASE DE DATOS — dónde viven los datos

- **Motor:** PostgreSQL autoalojado en un VPS y gestionado con Dokploy (compartido entre desarrollo y producción).
- **Definición:** `prisma/schema.prisma` describe las 37 tablas (Usuario, Producto, Insumo, Transaccion, CajaTurno, MovimientoCaja, CuentaCorriente, RegistroAuditoria, etc.).
- **Acceso:** exclusivamente vía Prisma Client. Ningún componente del navegador puede tocarla directamente.

**Analogía:** el *frontend* es el salón del restaurante (lo que ve el cliente), el *backend* es la cocina (donde se procesan los pedidos con reglas y controles), y la *base de datos* es la despensa/archivo (donde se guarda todo). El mesero (API) es el único que cruza entre el salón y la cocina; el cliente nunca entra a la despensa.

## 6. Cómo interactúan Frontend y Backend

La comunicación es siempre por **HTTP**, con un **token JWT** que identifica al usuario y viaja en una **cookie httpOnly** que el navegador adjunta automáticamente. Este es el viaje completo de una acción típica ("el cajero registra una venta"):

1. El cajero pulsa "Cobrar" en la pantalla

→ app/caja/venta/page.tsx (FRONT)

2. El componente llama a un hook

→ useRegistrarVenta() en hooks/caja.ts

3. El hook usa Axios para enviar POST /api/caja/venta

· el navegador adjunta la cookie httpOnly de sesión (seteada por /api/auth/login)

4. Llega al endpoint

→ app/api/caja/venta/route.ts (BACK)

· isRateLimited()

→ ¿demasiadas peticiones? (máx. 3 en 10s)

· requireAuth()

→ verifica el JWT y carga el usuario activo

· requireRole()

→ ¿es CAJERO/ADMIN/DUENO?

· VentaFisicaDTO.parse

→ valida los datos con Zod

5. El endpoint delega en la lógica de negocio

→ lib/server/caja/caja.service.ts

· recalcula el total EN EL SERVIDOR (no confía en el precio del navegador)

· descuenta stock por receta, registra el movimiento de caja, audita

· todo dentro de una TRANSACCIÓN (o se hace todo, o no se hace nada)

6. Prisma traduce a SQL y escribe en PostgreSQL

→ lib/prisma.ts

7. La respuesta JSON regresa al navegador; React Query actualiza la pantalla
- Las piezas que hacen esto posible
- | Pieza              | Archivo                    | Función                                                                                                                                                              |
|--------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cliente HTTP       | hooks/api.ts               | Axios. La sesión viaja sola en la cookie httpOnly (no hace falta interceptor de envío); un interceptor de respuesta redirige al login si recibe 401.                 |
| Caché de datos     | lib/providers.tsx          | React Query cachea respuestas, evita peticiones duplicadas y refresca datos automáticamente.                                                                         |
| Guardia de rutas   | lib/Guard.tsx              | ProtectedRoute envuelve páginas privadas: comprueba el token y el rol antes de mostrarlas (protección de experiencia, no de seguridad — la real está en el backend). |
| Sesión de servidor | lib/server/auth/session.ts | requireAuth lee el token (del header o de una cookie), lo verifica y carga el usuario. requireRole aplica los permisos.                                              |
- Detalle de seguridad importante:** el precio y el total de una venta **siempre se recalculan en el servidor** a partir de los precios reales de la base. Aunque alguien manipule el navegador para "pagar" un producto a 0 Bs, el backend ignora ese dato y usa el precio verdadero. Nunca se confía en el cliente.
- file:///C:/Users/alfre/Documents/Tarts/Trabajo/Elevate/elevateNext/docs/ARQUITECTURA\_ELEVATE.html

8/13



## 7. Webhooks e integraciones externas

---

El sistema es mayormente auto-contenido, pero tiene varios puntos de integración con el exterior:

### 7.1 Enlace del repartidor (patrón tipo webhook por token)

Cuando un pedido sale a delivery, el sistema genera un `driver_link_id` (un identificador corto e impredecible). El repartidor abre `app/driver/[token]` desde su celular, sin necesidad de tener cuenta. Ese token le da acceso *solo* a ese pedido, y le permite:

- Ver los datos del pedido: `GET /api/pedidos/driver/[token]`
- Actualizar el estado (EN\_CAMINO → LLEGÓ → ENTREGADO): `POST` del mismo endpoint
- Enviar su ubicación GPS en vivo ( `driver_lat` , `driver_lng` ), que el admin ve en el mapa Leaflet.

Es un patrón "webhook-like": una URL con un token secreto reemplaza al login para un actor externo de un solo uso.

### 7.2 Almacenamiento de imágenes (Vercel Blob)

Las imágenes de productos no se guardan en la base de datos (sería lento y caro), sino en **Vercel Blob**, un almacenamiento de archivos en la nube. La base solo guarda la URL. El endpoint `app/api/admin/upload` gestiona la subida.

### 7.3 Alertas de inventario (WhatsApp — simulado/preparado)

Existe un servicio de alertas ( `lib/server/alertas/whatsapp.service.ts` ) que, cuando los insumos bajan del umbral crítico, prepara y registra notificaciones (por ahora en modo simulado, listo para conectar un proveedor real de WhatsApp). Los envíos quedan registrados en la tabla `RegistroAlerta` .

### 7.4 Sondeo en tiempo real (polling)

Para "tiempo real" sin la complejidad de WebSockets, el frontend usa **polling**: el hook `useOrderPolling.ts` vuelve a preguntar por pedidos nuevos cada pocos segundos. Así el panel de caja/admin muestra pedidos entrantes y el mapa de repartidores se actualiza sin recargar la página.

## 8. Ciberseguridad: capas, dónde y cómo

---

La seguridad está implementada en **capas** (defensa en profundidad). Cada petición sensible atraviesa varios controles antes de tocar la base de datos:

| #  | Control                                    | Dónde                                              | Cómo protege                                                                                                                                                      |
|----|--------------------------------------------|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <b>Contraseñas cifradas</b>                | <code>lib/auth.ts</code>                           | bcrypt con "salt" (10 rondas). Las contraseñas nunca se guardan en texto plano; ni el administrador puede leerlas.                                                |
| 2  | <b>Tokens JWT firmados</b>                 | <code>lib/auth.ts</code>                           | Al iniciar sesión se emite un token firmado con un secreto. Sin ese secreto nadie puede falsificar una identidad. Expira a las 20 horas.                          |
| 3  | <b>Verificación por endpoint</b>           | <code>session.ts</code> → <code>requireAuth</code> | Cada endpoint privado revalida el token y confirma que el usuario existe y está <i>activo</i> en la base (un usuario desactivado pierde acceso al instante).      |
| 4  | <b>Control de roles (RBAC)</b>             | <code>requireRole</code>                           | Cada endpoint declara qué roles pueden usarlo (DUENO, ADMIN, CAJERO, CLIENTE). Un cajero no puede entrar a funciones de dueño.                                    |
| 5  | <b>Validación de entrada</b>               | <code>lib/server/dto/*</code> (Zod)                | Todo dato entrante se valida contra un esquema estricto (tipos, rangos, longitudes máximas). Rechaza datos malformados o maliciosos antes de procesarlos.         |
| 6  | <b>Límite de peticiones</b>                | <code>rate-limit</code>                            | Frena abusos: p. ej. máximo 3 ventas en 10 segundos. Mitiga ataques de fuerza bruta y duplicados.                                                                 |
| 7  | <b>Recálculo en servidor</b>               | <code>caja.service.ts</code>                       | Precios y totales se calculan con datos de la base, nunca con lo que envía el navegador. Impide manipulación de precios.                                          |
| 8  | <b>Lista blanca de campos</b>              | <code>pedidos/[id]/route.ts</code>                 | Al actualizar un registro solo se aceptan campos explícitamente permitidos. Previene "mass assignment" (que alguien inyecte campos que no debería).               |
| 9  | <b>Bitácora de auditoría</b>               | <code>audit.service.ts</code>                      | Cada acción sensible (venta, apertura/cierre de caja, cambios) se registra con usuario, rol, IP y navegador en una tabla <i>solo-append</i> . Trazabilidad total. |
| 10 | <b>Protección contra inyección SQL</b>     | Prisma                                             | Prisma usa consultas parametrizadas por diseño. No se concatena SQL a mano, así que la inyección SQL clásica no aplica.                                           |
| 11 | <b>Aislamiento de paquetes de servidor</b> | <code>next.config.ts</code>                        | bcrypt y Prisma se marcan como "externos del servidor": nunca se envían al navegador, evitando filtrar lógica sensible.                                           |

## Cómo se ve en la práctica (un endpoint real)

```
export async function POST(req) {
  if (isRateLimited(req, 10000, 3)) // 6. límite de peticiones
    return 429;
  const session = await requireAuth(req); // 3. verifica identidad (JWT)
  requireRole(session, ['CAJERO', 'DUENO', 'ADMIN']); // 4. permisos por rol
  const dto = VentaFisicaDTO.parse(body); // 5. valida entrada (Zod)
  const venta = await caja.registrarVentaFisica(session, dto, {ip, userAgent});
  // ↑ dentro: recálculo en servidor (7) + auditoría con IP (9), todo en transacción
}
```

**Fortaleza:** la seguridad real vive en el *backend*. El guardia del frontend ( `ProtectedRoute` ) solo mejora la experiencia (no muestra pantallas que no corresponden), pero aunque alguien lo saltee, el backend rechaza cualquier petición sin token válido y rol correcto. Esto es lo correcto: *nunca confiar en el cliente*.

## 9. Flujos completos de ejemplo

### 9.1 Venta en caja (mostrador)

```

Cajero abre turno → agrega productos al carrito → elige EFECTIVO/QR → Cobrar
↓
POST /api/caja/venta → valida → recalcula total → crea Transaccion
↓
descuenta stock de insumos por receta (MovimientoInterno)
↓
registra MovimientoCaja + actualiza saldo de la cuenta (EFECTIVO o QR)
↓
audita la operación → responde → el dashboard refleja la venta al instante

```

### 9.2 Pedido web con delivery

```

Cliente arma pedido en la tienda → POST /api/pedidos (crea Transaccion PENDIENTE)
↓
El cajero lo ve por polling → lo prepara → genera enlace de repartidor (token)
↓
Repartidor abre app/driver/[token] → actualiza estado + envía GPS
↓
Admin ve la ubicación en el mapa (Leaflet) → ENTREGADO → se descuenta stock y se cierra

```

### 9.3 Cierre de caja

```

Cajero cuenta el dinero físico → POST /api/caja/cierre (real_efectivo, real_qr)
↓
El sistema calcula ESPERADO = apertura + ventas - gastos
↓
Compara ESPERADO vs REAL → registra la DIFERENCIA (sobrante/faltante) por método
↓
Marca el turno CERRADO y lo archiva en el historial (auditado)

```

## 10. Despliegue (deploy)

El proyecto está preparado para desplegarse en **Vercel** (la plataforma nativa de Next.js). El proceso, definido en `vercel.json`, hace: instalar dependencias → generar el cliente Prisma → construir la app. **El build no toca la base de datos**: los cambios de esquema se aplican por separado, mediante migraciones SQL aditivas revisadas y registradas con `prisma migrate resolve`.

Alternativamente, la caja puede correr como **app de escritorio** empaquetada con Electron (`npm run electron:build`), que abre la web en una ventana nativa de Windows.

# 11. Riesgos detectados y recomendaciones

Durante la revisión se identificaron puntos que conviene atender. Se ordenan por severidad:

## ● RESUELTO — el build de despliegue ya no modifica la base de datos

Versiones anteriores del despliegue ejecutaban `prisma db push --accept-data-loss` y el seed en cada build, lo que podía sobrescribir estructura y datos de producción. El `vercel.json` actual solo genera el cliente Prisma y construye la aplicación; los cambios de esquema se aplican de forma controlada y manual (migraciones SQL aditivas + `prisma migrate resolve`). **Importante:** verificar que el panel de Vercel no tenga un *build command* propio que sobrescriba al de `vercel.json`.

## ● ALTO — Secretos en `.env`

El archivo `.env` contiene la URL de la base (con usuario y contraseña) y el secreto JWT en texto plano. Debe estar **siempre** en `.gitignore` y nunca subirse al repositorio. Si alguna vez se subió, esas credenciales deben rotarse (cambiarse). El secreto JWT actual es corto y predecible; conviene reemplazarlo por uno largo y aleatorio.

## ● RESUELTO — Sesión en cookie `httpOnly`

El token de sesión ya no se guarda en `localStorage`: el servidor lo entrega en una cookie `httpOnly` + `secure` + `sameSite` (`lib/server/auth/cookies.ts`), inaccesible para el JavaScript del navegador, lo que mitiga el robo de sesión por XSS. `requireAuth` la lee en cada petición.

## ● MEDIO — Token del repartidor corto

El `driver_link_id` es un fragmento de UUID (8 caracteres). Para un enlace de bajo valor y corta vida es aceptable, pero conviene usar un token más largo y con expiración para evitar que se adivine.

## ● MENOR — Base de datos compartida sin coordinación de migraciones

Desarrollo y producción usan la misma base, y distintas ramas han aplicado esquemas diferentes. Se recomienda separar la base de desarrollo de la de producción, y acordar que los cambios de esquema siempre pasen por migraciones revisadas (nunca `db push` directo).